Исследование аптечного сектора

Исследование выполнила Коробкова Мария
в рамках курса по анализу данных

Run SQL.73 | PostgreSQL.74 | PostgreSQL.75 | PostgreSQL.76

```sql
-- Запрос "Топ-3 аптеки"
-- Объём продаж = price * count

SELECT
    pharmacy_name,
    SUM(price * COUNT) AS total_sales
FROM pharma_orders
GROUP BY pharmacy_name
ORDER BY total_sales DESC
LIMIT 3;
```

pharmacy_name	total_sales
Столичная	1896483
Доктор Айболит	1893776
Здравсити	1863037

Лидеры среди аптек по объёму продаж

Использованные функции:

SUM() — суммирует значения
price * COUNT — вычисляет объём продаж (цена × количество)
GROUP BY — группирует данные по аптекам
ORDER BY ... DESC — сортирует по убыванию
LIMIT 3 — ограничивает вывод топ-3

Вывод по данным:

Топ-3 аптеки по объёму продаж очень близки по показателям (разница менее 2%). Лидирует "Столичная" с результатом 1,896 млн, за ней следуют "Доктор Айболит" (1,894 млн) и "Здравсити" (1,863 млн). Конкуренция высокая, все три аптеки показывают схожие результаты.

Лидеры среди лекарств по объёму продаж

Run SQL.73 | PostgreSQL.74 | PostgreSQL.75 | PostgreSQL.7

```sql
-- Запрос "Топ-3 лекарства"

SELECT
    drug,
    SUM(price * COUNT) AS total_sales
FROM pharma_orders
GROUP BY drug
ORDER BY total_sales DESC
LIMIT 3;
```

drug	total_sales
бупрофен	777548
итрамон	776332
квафорте	771579

Использованные функции:

SUM() — суммирует значения
price * COUNT — вычисляет объём продаж (цена × количество)
GROUP BY — группирует данные по лекарствам
ORDER BY ... DESC — сортирует по убыванию
LIMIT 3 — ограничивает вывод топ-3

Вывод по данным:

Топ-3 лекарства также показывают очень близкие результаты по объёму продаж (разница менее 1%). Лидирует "Ибупрофен" с результатом 777 548, затем "Цитрамон" (776 332) и "Аквафорте" (771 579). Все три препарата пользуются схожим спросом, что говорит о стабильном потреблении популярных лекарств.

Run SQL.73 | PostgreSQL.74 | PostgreSQL.75 | PostgreSQL.76

```sql
-- Запрос "Аптеки от 1.8 млн оборота".
--Фильтрация агрегированных значений.

SELECT
    pharmacy_name,
    SUM(price * COUNT) AS total_sales
FROM pharma_orders
GROUP BY pharmacy_name
HAVING SUM(price * COUNT) > 1800000
ORDER BY total_sales DESC;
```

pharmacy_name	total_sales
Столичная	1896483
Доктор Айболит	1893776
Здравсити	1863037

Фильтрация аптек с оборотом выше 1.8 млн рублей

Использованные функции:

SUM() — суммирует значения
price * COUNT — вычисляет объём продаж (цена × количество)
GROUP BY — группирует данные по аптекам
HAVING — фильтрует агрегированные результаты (оставляет только аптеки с оборотом > 1.8 млн)
ORDER BY ... DESC — сортирует по убыванию

Вывод по данным:

Три аптеки ("Столичная" — 1,896 млн, "Доктор Айболит" — 1,894 млн, "Здравсити" — 1,863 млн) превысили порог незначительно и показывают близкие результаты с разница менее 2%. Это указывает на высокую конкуренцию и схожие объёмы продаж.

Run SQL.73 | PostgreSQL.74 | PostgreSQL.75 | PostgreSQL.76 | PostgreSQL.77 | Post

```sql
-- Запрос "Накопленная сумма продаж по каждой аптеке"
--Оконная функция для расчета накопленного значения.
--Поля: аптека, дата заказа, накопленная сумма.


SELECT
    pharmacy_name,
    report_date,
    SUM(price * COUNT) AS daily_sales,
    SUM(SUM(price * COUNT)) OVER (
        PARTITION BY pharmacy_name
        ORDER BY report_date
        ROWS UNBOUNDED PRECEDING
    ) AS cumulative_sales
FROM pharma_orders
GROUP BY pharmacy_name, report_date
ORDER BY pharmacy_name, report_date;
```

pharmacy_name	report_date	daily_sales	cumulative_sales
Столичная	2024-05-28	22494	1807639
Столичная	2024-05-29	21805	1829444
Столичная	2024-05-30	20146	1849590
Столичная	2024-05-31	18461	1868051
Столичная	2024-06-01	28432	1896483
Аптека №1	2024-02-01	18754	18754
Аптека №1	2024-02-02	11949	30703
Аптека №1	2024-02-03	10391	41094
Аптека №1	2024-02-04	13807	54901
Аптека №1	2024-02-05	15105	70006
Аптека №1	2024-02-06	4850	74856
Аптека №1	2024-02-07	[illegible]	[illegible]

Динамика продаж по каждой аптеке

Использованные функции:

SUM(price * COUNT) — расчёт дневных продаж
SUM(...) OVER (PARTITION BY ...) — оконная функция для накопленной суммы
PARTITION BY pharmacy_name — разбивка по аптекам
ORDER BY report_date — сортировка по датам
ROWS UNBOUNDED PRECEDING — суммирование от первой строки до текущей

Вывод по данным:

Запрос показывает динамику продаж по каждой аптеке: дневную выручку и накопленную сумму. Видно, что "Столичная" за 5 дней (28.05-01.06) накопила 1 896 млн, а "Аптека №1" начиная с февраля постепенно наращивает оборот (к 6 февраля — 74 856). Удобно для отслеживания прогресса каждой аптеки во времени.

Run | PostgreSQL.76 | PostgreSQL.77 | PostgreSQL.78 | PostgreS

```sql
-- Запрос "Количество клиентов в аптеках".
/*Соединяем таблицы заказов с таблицами клиентов.
Подсчитываем количество уникальных клиентов на каждую аптеку и
сортируем аптеки по убыванию данного количества клиентов.*/

SELECT
    P.pharmacy_name,
    COUNT(DISTINCT P.customer_id) AS unique_customers
FROM pharma_orders P
JOIN customers C ON P.customer_id = C.customer_id
GROUP BY P.pharmacy_name
ORDER BY unique_customers DESC;
```

pharmacy_name	unique_customers
Столичная	441
Аптека.ру	437
Доктор Айболит	430
Здравсити	428
Аптека №1	427
Горздрав	426

Количество уникальных клиентов

Использованные функции:

COUNT(DISTINCT customer_id) — подсчёт уникальных клиентов
JOIN — соединение таблиц заказов и клиентов
GROUP BY — группировка по аптекам
ORDER BY ... DESC — сортировка по убыванию количества клиентов

Вывод по данным:

Все аптеки имеют очень близкое количество уникальных клиентов (разница всего 15 клиентов между первой и последней). Конкуренция за клиентов крайне высокая, все аптеки показывают схожие результаты по охвату аудитории.

Run | PostgreSQL.76 | PostgreSQL.77 | PostgreSQL.78 | PostgreSQL.79 | PostgreSQL.80

```sql
-- Запрос "Лучшие клиенты".
/*Соединяем таблицы клиентов и заказов.
Подсчитываем общую сумму заказов для каждого клиента.
Ранжируем клиентов и выводим топ-10. Учитываем имя, фамилию и отчество.*/


WITH customer_totals AS (
    SELECT
        C.customer_id,
        C.last_name,
        C.first_name,
        C.second_name,
        SUM(P.price * P.count) AS total_customers_purchases
    FROM pharma_orders P
    JOIN customers C ON P.customer_id = C.customer_id
    GROUP BY C.customer_id, C.last_name, C.first_name, C.second_name
),
ranked_customers AS (
    SELECT
        last_name,
        first_name,
        second_name,
        total_customers_purchases,
        ROW_NUMBER() OVER (ORDER BY total_customers_purchases DESC) AS rn
    FROM customer_totals
)
SELECT
    rn,
    last_name,
    first_name,
    second_name,
    total_customers_purchases
FROM ranked_customers
WHERE rn <= 10;
```

rn	last_name	first_name	second_name	total_customers_purch
1	Буров	Бажен	Германович	45174

Лучшие клиенты по сумме покупок

Использованные функции:

SUM(price * count) — расчёт общей суммы заказов клиента
JOIN — соединение таблиц заказов и клиентов
GROUP BY — группировка по клиентам
ROW_NUMBER() OVER (ORDER BY ...) — оконная функция для ранжирования клиентов
CTE (WITH) — общие табличные выражения для структурирования запроса
WHERE rn <= 10 — фильтрация топ-10

Вывод по данным:

Запрос выявляет лучших клиентов по сумме покупок. Все клиенты из топ-10 — ценные покупатели с оборотом выше 38 тысяч. Лидер — Буров Бажан Германович с результатом 45 174. CTE позволяют сначала подсчитать суммы по каждому клиенту, затем пронумеровать их по убыванию и вывести топ-10 с полными ФИО. Удобно для анализа VIP-клиентов.

Run | PostgreSQL.77 | PostgreSQL.78 | PostgreSQL.79 | PostgreSQL.80 | PostgreSQL.81

```sql
-- Запрос "Накопленная сумма по клиентам"
/*Соединить таблицы заказов и клиентов
Соединить ФИО в одно поле
Рассчитать накопленную сумму по каждому клиенту*/
-- Накопленная сумма покупок по каждому клиенту во времени
-- Объединяем ФИО в одно поле и используем оконную функцию

SELECT
    CONCAT_WS(' ', C.last_name, C.first_name, C.second_name) AS full_name,
    P.report_date,
    P.price * P.count AS order_amount,
    SUM(P.price * P.count) OVER (
        PARTITION BY C.customer_id
        ORDER BY P.report_date
    ) AS cumulative_spent
FROM pharma_orders P
JOIN customers C ON P.customer_id = C.customer_id
ORDER BY full_name, P.report_date;
```

full_name	report_date	order_amount	cumulative_spent
Алексеев Сократ Федосьевич	2024-04-02	2691	13081
Алексеев Сократ Федосьевич	2024-04-07	1275	14356
Алексеев Сократ Федосьевич	2024-04-17	1416	15772
Алексеев Сократ Федосьевич	2024-04-28	1665	17437
Алексеев Сократ Федосьевич	2024-05-13	1068	18505
Алексеев Сократ Федосьевич	2024-05-27	1220	19725
Андреев Тит Жоресович	2024-02-09	2883	2883
Андреев Тит Жоресович	2024-02-13	2250	5133
Андреев Тит Жоресович	2024-03-01	4490	9623
Андреев Тит Жоресович	2024-03-16	3408	13031
Андреев Тит Жоресович	2024-03-22	4460	17491

История покупок каждого клиента

Использованные функции:

CONCAT_WS(' ', ...) — объединение ФИО в одно поле

SUM() OVER (PARTITION BY ... ORDER BY ...) — оконная функция для расчёта накопленной суммы

PARTITION BY customer_id — разбивка по каждому клиенту

ORDER BY report_date — сортировка по датам для хронологии

JOIN — соединение таблиц заказов и клиентов

Вывод по данным:

Запрос показывает историю покупок каждого клиента с нарастающим итогом. Например, Алексеев Сократ Федосьевич с апреля по май сделал 6 покупок на общую сумму 19 725 (последнее значение cumulative_spent). Удобно отслеживать лояльность и активность клиентов во времени.

Run | PostgreSQL.78 | PostgreSQL.79 | PostgreSQL.80 | PostgreSQL.81 | Postg

```sql
-- Запрос "Самые частые клиенты аптек Горздрав и Здравсити"
/*Сделать две временные таблицы: для аптеки горздрав и здравсити (WITH)
Внутри каждой соединить таблицы заказов и клиентов (JOIN)
Внутри каждой привести данные в формат "клиент - кол-во заказов в аптеке"
Внутри каждой оставить топ 10 клиентов каждой аптеки
Объединить клиентов с помощью UNION*/

WITH gzd_clients AS (
    SELECT
        CONCAT_WS(' ', C.last_name, C.first_name, C.second_name) AS full_name,
        P.pharmacy_name,
        COUNT(*) AS order_count,
        ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS rn
    FROM pharma_orders P
    JOIN customers C ON P.customer_id = C.customer_id
    WHERE P.pharmacy_name = 'Горздрав'
    GROUP BY C.customer_id, C.last_name, C.first_name, C.second_name, P.pharmacy_name
),
zs_clients AS (
    SELECT
        CONCAT_WS(' ', C.last_name, C.first_name, C.second_name) AS full_name,
        P.pharmacy_name,
        COUNT(*) AS order_count,
        ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS rn
    FROM pharma_orders P
    JOIN customers C ON P.customer_id = C.customer_id
    WHERE P.pharmacy_name = 'Здравсити'
    GROUP BY C.customer_id, C.last_name, C.first_name, C.second_name, P.pharmacy_name
)
SELECT full_name, pharmacy_name, order_count
FROM gzd_clients
WHERE rn <= 10
UNION ALL
SELECT full_name, pharmacy_name, order_count
FROM zs_clients
WHERE rn <= 10
ORDER BY pharmacy_name, order_count DESC;
```

full_name	pharmacy_name	order_count
Беляев Клавдий Измаилович	Горздрав	8

Самые частые клиенты аптек Горздрав и Здравсити

Использованные функции:

CONCAT_WS(' ', ...) — объединение ФИО в одно поле

SUM() OVER (PARTITION BY ... ORDER BY ...) — оконная функция для расчёта накопленной суммы

PARTITION BY customer_id — разбивка по каждому клиенту

ORDER BY report_date — сортировка по датам для хронологии

JOIN — соединение таблиц заказов и клиентов

Вывод по данным:

Запрос показывает историю покупок каждого клиента с нарастающим итогом. Например, Алексеев Сократ Федосьевич с апреля по май сделал 6 покупок на общую сумму 19 725 (последнее значение cumulative_spent). Удобно отслеживать лояльность и активность клиентов во времени.

Спасибо за внимание